

哈爾濱工業大學
HARBIN INSTITUTE OF TECHNOLOGY

自然语言处理C实验III

利用对话数据微调基础大模型

——基于Qwen3-4B-Instruct-2507的中文医疗对话 QLoRA 的本地微调

计算学部人工智能学院

2026年秋季学期

一、实验简介

Qwen3-4B-Instruct-2507 是阿里云通义千问团队 2025 年发布的小尺寸指令模型，采用非思考模式，适合在消费级显卡上做指令微调。QLoRA 的做法是：把冻结的基座权重压到 4-bit NF4，只在 Transformer 线性层旁训练低秩 LoRA 矩阵。本实验的可训练参数约 1,650 万，占 4B 总参数的 0.41%，因此 8GB 显存可以完成训练，但余量很小，训练时需要关闭游戏、浏览器硬件加速等占用显存的程序。

实验数据为复旦大学数据智能与社会计算实验室公开的 DISC-Med-SFT 中文医疗对话数据。每条医生回复与紧邻的前一条用户提问配成一条单轮样本；训练集与验证集按对话级 9:1 切分。评测使用实验要求的两个测试集：医疗对话测试集 Chinese-medical-dialogue，用于评估真实医疗问答；通用知识测试集 CMMLU computer_science，用于观察微调对通用知识的影响。Chinese-medical-dialogue 取前 100 条作为 medical_100，CMMLU 使用 computer_science 的 204 道单选题；两者均不参与训练。

本指导书的参考实现使用 LLaMA-Factory 完成 QLoRA 训练，使用 Transformers + PEFT 完成批量推理；两者共用同一个 conda 环境。训练产物是 LoRA adapter，不合并进 4-bit 基座；推理时直接加载“4-bit 基座 + adapter”，以保证 Base 与 LoRA 两组使用完全相同的量化基座。

整体流程：安装环境 → 下载模型与数据 → 准备数据 → align100 训练 → Base/LoRA 推理 → ROUGE/CMMLU 统计 → 撰写报告。

学习参考文件：

1. LoRA 论文：https://arxiv.org/abs/2106.09685
2. DISC-Med-SFT 数据集：https://aistudio.baidu.com/dataset/detail/323449/intro
3. CMMLU 测试集：https://aistudio.baidu.com/dataset/detail/340353/intro
4. Chinese-medical-dialogue 测试集：https://aistudio.baidu.com/dataset/detail/323370/intro

二、实验目标

(1) 掌握使用领域对话数据对基础大模型做 QLoRA 微调的基本方法，理解 4-bit 量化基座与 LoRA 适配器的关系。

(2) 了解指令微调数据的构造：多轮对话如何单轮化、训练/验证如何切分、ShareGPT 格式如何注册。

(3) 熟悉医疗对话生成结果的自动评测：ROUGE-1/2/L 的计算口径。

(4) 能够对比微调前后的领域生成质量与通用知识变化，并结合显存、训练损失和生成案例分析解释结果。

三、 实验内容

3.1 硬件与显存说明

本实验需要 NVIDIA GPU，显存至少 8GB。RTX 50 系列（如 RTX 5060、5070）属于 sm120，必须使用 CUDA 12.8 及以上的 PyTorch；其它显卡使用与驱动匹配的 CUDA 版本。

8GB 是参考配置的下限。Qwen3-4B 的 4-bit 权重约 2.5GB，其余显存用于 LoRA 参数与优化器状态、序列长度 1024 的激活值和输出层 logits；参考机器整卡峰值约 7.5GB，空闲余量只有 0.3 - 0.5GB。

训练时出现 CUDA out of memory（OOM），按以下顺序处理：

关闭占用显存的程序。游戏、浏览器硬件加速、视频播放和桌面特效都会占用显存，训练前全部关闭；释放后 nvidia-smi 的已用显存应只剩系统桌面占用。

调小 cutoff_len。cutoff_len 位于 configs\qwen3_4b_qlora_align100.yaml，表示单条训练样本（提问与回复拼接后）的最大 token 长度，超出部分会被截断。当前值为 1024，OOM 时改为 512，激活显存约减半。llamafactory-cli train configs\qwen3_4b_qlora_align100.yaml 会读取该配置，不需要在命令行单独传入。

改用更小的基座模型。以上调整仍不足时，换用 3B 级模型，并同步修改 model_name_or_path、lora_target 等与模型有关的配置。

3.2 软件版本（参考实现）

软件	版本	用途
Python	3.11.17	conda 虚拟环境
PyTorch	2.7.1+cu128	GPU 训练与推理
torchvision / torchaudio	0.22.1 / 2.7.1	LLaMA-Factory 依赖
bitsandbytes	0.50.2	4-bit NF4 量化与分页优化器
LLaMA-Factory	0.9.5	QLoRA 训练
Transformers	5.6.0	模型加载与生成
PEFT	0.18.1	LoRA adapter 加载
ModelScope	最新稳定版	下载 Qwen3-4B 模型

版本不必完全一致，如果使用更新版本，先确认单步训练能正常启动，再开始完整训练。

3.3 路径约定

本指导书用两个占位符表示本机路径。使用时请替换成自己的实际路径；路径中尽量不要有空格和中文，以免 Windows 下的编码与转义问题。

占位符	说明
<PROJECT_ROOT>	项目根目录，放数据、模型、脚本、训练产物
<CONDA_ENV>	conda 虚拟环境目录

3.4 GPU 与 CUDA 自检

在 PowerShell 中先确认驱动和显卡可见，再创建环境。RTX 50 系列（5060/5070 等）是 sm120，必须安装 CUDA 12.8 及以上的 PyTorch 版本；cu126 可能缺少 sm120 内核。

```
nvidia-smi
# 期望看到 GPU 名称、显存、Driver Version
```

安装 PyTorch 后必须做三项验收：CUDA 可用、CUDA 构建版本、算力。三项都正确才继续。

```
python -c "import torch; print(torch.__version__); print(torch.version.cuda);
print(torch.cuda.is_available()); print(torch.cuda.get_device_capability())"
# 参考输出：
# 2.7.1+cu128
# 12.8
# True
# (12, 0)   # RTX 5060 为 sm120
```

3.5 Anaconda / Miniconda

如果本机已经安装 Anaconda，可跳过本节。否则从 Anaconda 官网或国内镜像安装 Miniconda/Anaconda，安装后打开 Anaconda Prompt 或 PowerShell，确认 conda 可用。

内容	地址/方式	验证
Anaconda	https://www.anaconda.com/download	conda --version
Miniconda	https://docs.conda.io/en/latest/miniconda.html	conda --version
国内镜像	https://mirrors.tuna.tsinghua.edu.cn/anaconda/miniconda/	安装后 conda init

建议使用 conda 的 --prefix 参数把环境建在空间充足的位置，而不是依赖默认 envs_dirs。下面的命令以 <CONDA_ENV> 为例：

```
conda create --prefix <CONDA_ENV> python=3.11 -y
conda activate <CONDA_ENV>
where python
# 必须指向 <CONDA_ENV>\python.exe
```

如果 conda activate 报错，先在 PowerShell 执行 conda init powershell 并重开终端，或直接使用 Anaconda Prompt。不要把环境建到 C 盘用户目录，后面模型和数据会占用较多空间。

3.6 PyTorch（GPU 版）

在已激活的 <CONDA_ENV> 中，根据显卡安装对应的 GPU 版 PyTorch。RTX 50 系列使用 cu128；其它显卡选择与驱动匹配的 CUDA 版本。参考实现使用 PyTorch 2.7.1 + cu128。

```
python -m pip install --upgrade pip setuptools wheel
python -m pip install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cu128
python -c "import torch; print(torch.__version__, torch.version.cuda,
torch.cuda.is_available(), torch.cuda.get_device_capability())"
```

安装包约 3GB，官方源较慢时可以换用镜像或稍后重试。

3.7 Python 依赖

依次安装 bitsandbytes、LLaMA-Factory、ModelScope 和评测依赖。LLaMA-Factory 直接从 PyPI 安装即可，不必克隆 GitHub；如果后续需要查看源码或自定义，再参考官方仓库。

```
python -m pip install bitsandbytes==0.50.2
python -m pip install "llamafactory==0.9.5"
python -m pip install modelscope rouge-chinese jieba
python -c "import bitsandbytes as bnb; print('bnb', bnb.__version__)"
llamafactory-cli version
```

如果 bitsandbytes 导入时报 DLL 缺失或找不到 GPU，说明它与 CUDA 版本不匹配，需要换用支持 CUDA 12.8/12.9 的版本。

3.8 基座模型 Qwen3-4B-Instruct-2507

使用 ModelScope 下载模型，--local_dir 指向自己的模型目录；下面的命令以 <PROJECT_ROOT>\models 为例。模型约 7.5GB，包含 3 个 safetensors 分片；下载中断后重新执行同一条命令会续传。

```
modelscope download --model Qwen/Qwen3-4B-Instruct-2507 `
  --local_dir <PROJECT_ROOT>\models\Qwen3-4B-Instruct-2507
```

下载后检查目录中至少有 config.json、tokenizer.json、tokenizer_config.json、model.safetensors.index.json 和 3 个 .safetensors 文件；分片总大小应在 7.4GiB 左右。

文件/检查项	期望
config.json	architectures 包含 Qwen3ForCausalLM
model-00001-of-00003.safetensors	约 3.69GiB
model-00002-of-00003.safetensors	约 3.71GiB
model-00003-of-00003.safetensors	约 0.09GiB
tokenizer.json	约 11MB

3.9 数据集

三个数据集都从百度飞桨 AI Studio 的公开数据集页面下载，下载前需要登录百度账号。

数据集	用途	下载地址
DISC-Med-SFT	训练与验证	https://aistudio.baidu.com/dataset/detail/323449/intro
Chinese-medical-dialogue	医疗对话测试集	https://aistudio.baidu.com/dataset/detail/323370/intro
CMMLU（computer_science）	通用知识测试集	https://aistudio.baidu.com/dataset/detail/340353/intro

3.10 准备训练集和验证集

参考脚本 **prepare_sharegpt.py** 。读取 DISC-Med-SFT 原始 JSONL，完成过滤、对话级 9:1 切分、单轮化、抽样，输出数据集文件和 dataset_info.json。输入文件与输出目录换成自己的实际路径。

```
cd <PROJECT_ROOT>
python scripts\prepare_sharegpt.py `
  --src data\raw\DISC-Med-SFT_released.jsonl `
  --out-dir data\processed `
  --train-n 20000 --val-n 1000 --subset-n 2000
```

输出文件	条数	用途
train_2k_sharegpt.json	2,000	100 步主实验，100×16=1,600 条
val_1k_sharegpt.json	1,000	验证/监控用
dataset_info.json	2 个条目	LLaMA-Factory 数据集注册

3.11 准备两个测试集

Chinese-medical-dialogue 与 CMMLU 下载后是原始评测格式，需要转换为评测文件，参考脚本**prepare_medical.py**和**prepare_cmmlu.py**。--src 与 --out-dir 换成自己的实际路径：

```
cd <PROJECT_ROOT>
python scripts\prepare_medical.py --src data\raw\train_0001_of_0001.json --out-dir data\eval
python scripts\prepare_cmmlu.py --src data\raw\computer_science.csv --out-dir data\eval
```

prepare_medical.py 取前 100 条，输出 medical_100.jsonl；prepare_cmmlu.py 转换 computer_science 的全部 204 题，输出 cmmlu_computer_science.jsonl。

3.12 输出格式与 dataset_info.json

两个数据文件都是 JSON 数组，每个元素是一条 ShareGPT 样本；human 对应用户提问，gpt 对应医生回复。LLaMA-Factory 通过 dataset_info.json 中的 formatting、columns 和 tags 读取这些字段。

```
{
  "conversations": [
    {"from": "human", "value": "有血压高史 可控制 有时候头疼"},
    {"from": "gpt", "value": "您好，很高兴为您提供帮助。请问除了头疼之外……"}
  ]
}
```

3.13 启动 100 步训练

本实验参考配置为**qwen3_4b_qlora_align100.yaml**：使用 train_2k 数据集中的 2,000 条样本，等效批 16，共训练 1,600 条样本；LoRA rank 8，学习率 1e-4，cosine 调度，warmup 20，seed 23。参考机器上 100 步约 12 分钟，整卡峰值约 7.5GB。

参数	取值	说明
dataset	med_train_2k	2,000 条训练子集
max_samples	2000	使用全部 2,000 条
max_steps	100	100 步 × 等效批 16 = 1,600 条样本
save_steps	50	第 50、100 步保存 adapter
warmup_steps	20	学习率预热
output_dir	自定义	训练输出目录，由自己指定
overwrite_output_dir	false	保留每次 run

```
cd <PROJECT_ROOT>
llamafactory-cli train configs\qwen3_4b_qlora_align100.yaml 2>&1 | Tee-Object -FilePath align100.log
echo "exit=$LASTEXITCODE"
```

3.14 训练参数解释

参数	取值	作用/说明
quantization_bit / method	4 / bnb	基座用 bitsandbytes NF4 量化
quantization_type	nf4	正态分布权重的最优 4-bit 类型
double_quantization	true	再量化量化常数，进一步省显存
lora_rank / alpha	8 / 16	低秩矩阵大小；8GB 上不建议调到 16
lora_target	all	所有线性层注入 LoRA
cutoff_len	1024	最大序列长度；OOM 时降到 512
per_device_train_batch_size	1	8GB 上保持 1
gradient_accumulation_steps	16	等效批 16，不增加激活显存
optim	paged_adamw_8bit	分页 8-bit 优化器，缓解显存尖峰
gradient_checkpointing	true	用重算换激活显存
bf16 / fp16	true / false	RTX 5060 支持 bf16；老卡按硬件调整
flash_attn	sdpa	Windows 不装 flash-attn，用 PyTorch SDPA

3.15 Base 与 LoRA 的关系

训练产出的是 LoRA adapter，不是完整模型。推理时加载同一个 4-bit Qwen3-4B 基座：**Base 组不加载 adapter，LoRA 组加载 adapter**。两组必须使用同一基座、同一 prompt、同一解码参数。不要把 adapter 合并进 4-bit 基座，**本实验不需要合并**。

组别	基座	adapter	用途
Base	4-bit Qwen3-4B	不加载	微调前基线
LoRA100	4-bit Qwen3-4B	训练输出目录\checkpoint-100	微调后 qlora 检查点

3.16 批量推理

批量推理参考 **gen_peft.py** 。它读取测试集文件中的题目与参考答案，套用 Qwen3 的 chat template，关闭 thinking，4-bit NF4 加载，可选加载 PEFT adapter，贪心生成并写出预测结果。模型、测试集与输出路径换成自己的目录。

```
# Base: 不加载 adapter
python scripts\gen_peft.py
  --model <PROJECT_ROOT>\models\Qwen3-4B-Instruct-2507
  --adapter none `
  --input data\eval\medical_100.jsonl
  --output outputs\eval\base\medical_100.jsonl
  --max-new-tokens 256 --batch-size 4

# LoRA: 加载训练好的 adapter
python scripts\gen_peft.py
  --model <PROJECT_ROOT>\models\Qwen3-4B-Instruct-2507
  --adapter <训练输出目录>\checkpoint-100
  --input data\eval\medical_100.jsonl
  --output outputs\eval\lora100\medical_100.jsonl
  --max-new-tokens 256 --batch-size 4
```

batch-size 4 在 8GB 上可用；如果 OOM，降到 2 或 1。

3.17 CMMLU 的统一作答约束

Qwen3-4B 在原始题干下倾向于输出长篇解释，旧解析器会把题干里的选项字母误当成答案，导致大量“未解析”。本实验给 Base 和 LoRA 使用同一条 system prompt，要求只输出选项字母：

```
你是一名计算机科学考试助手。只输出正确选项的字母（A/B/C/D），不要解释，不要重复题干。
```

在参考脚本**gen_peft.py**中，系统提示词通过 --system-prompt 参数直接传入，下面的命令以 Base 组为例：

```
python scripts\gen_peft.py `
  --model <PROJECT_ROOT>\models\Qwen3-4B-Instruct-2507 `
  --adapter none `
  --input data\eval\cmmlu_computer_science.jsonl `
  --output outputs\eval\base\cmmlu.jsonl `
  --max-new-tokens 32 --batch-size 4 `
  --system-prompt "你是一名计算机科学考试助手。只输出正确选项的字母（A/B/C/D），不要解释，
不要重复题干。"
```

3.18 指标统计

医疗对话计算 ROUGE-1/2/L，参考脚本为 **evaluate_medical.py** 。CMMLU计算正确率，参考脚本为 **evaluate_cmmlu.py** 。下面的输出路径换成自己的目录：

```
# 医疗: ROUGE
python scripts\evaluate_medical.py --pred outputs\eval\base\medical_100.jsonl --
save outputs\reports\base_medical.json
python scripts\evaluate_medical.py --pred outputs\eval\lora100\medical_100.jsonl
--save outputs\reports\lora_medical.json

# CMMLU: 正确率 + 错例
python scripts\evaluate_cmmlu.py --pred outputs\eval\base\cmmlu.jsonl --save
outputs\reports\base_cmmlu.json --wrong-file
outputs\reports\base_cmmlu_wrong.jsonl
python scripts\evaluate_cmmlu.py --pred outputs\eval\lora100\cmmlu.jsonl --save
outputs\reports\lora_cmmlu.json --wrong-file
outputs\reports\lora_cmmlu_wrong.jsonl
```

四、 实验报告{计5/20分}

按照学术论文标准撰写梳理自己的工作，凝练自己的核心发现（贡献），巧妙的将上述实验结果，包括的实现及评估过程总结、心得，融入其中；

避免流水账，4页左右为好，最长不应超过8页

实验报告在说明问题时建议加入实例作为辅助说明；

请确保实验报告格式清晰、一致，内容的条理性和完整性。

五、 评分方式

1）该实验成绩满分20分：代码成绩15分，报告成绩5分；

2）实现LoRA微调基础大模型，分析所用到的LoRA各项参数，并对LoRA微调后的模型和原模型的实验结果进行比对分析：记为12分；

3）采用调整LoRA参数的方式，不改变训练语料，提升微调后的大模型水平，记为13分；

备注：调参次数应该提供的算力额度内完成，请预先调研相关资料，制订调参计划。

4）查阅文献，利用除LoRA微调外的其他大模型微调方法，实现该实验，并提升大模型水平，计满15分；

备注：注意在算力额度限制内完成。

5）报告成绩:5分。

学术论文写作风格，避免实验过程堆砌；

内容完整、格式规范；

包含所使用的参考文献[重要]。

注意：如果不提交实验报告，或者只有实验报告而无代码，本次实验成绩清零；在完成该实验后，如果实验报告表述和实验内容严重不一致，缺少关键内容，本次实验总成绩直接降为12分。

六、 提交内容

实验三需提交的材料如下，全部放入一个压缩包（压缩包内含实验报告与code 文件夹）：

数据准备：训练数据处理脚本、医疗测试集处理脚本、CMMLU处理脚本

训练：训练配置文件、训练运行截图

推理与评测：批量推理脚本、推理运行截图、Chinese-medical-dialogue评测脚本、CMMLU评测脚本，评测结果文件及截图

报告：实验报告（截图放进报告里）

七、 提交方式

实验采用金山云文档（WPS）的方式进行实验文件的提交，链接如下：

https://yunbiz.wps.cn/c/collect/cfGrAxSXKoB

提交内容为每个实验单独一个压缩包，在表单内正确填写“实验x、姓名、学号”会自动命名。

由于有三个实验，所以需要上传三次文件，分别为实验一、实验二、实验三的压缩包。

压缩包内包含实验报告和命名为code的文件夹，code文件夹用于存放代码文件。